Lesson 17 - Homework

Comprehensions, Iterators və Generators

Mövzular: List Comprehension, Dictionary Comprehension, Set Comprehension, `if...else`, Iterable, Iterator, `iter()`, `next()`, `StopIteration`, Generator, `yield`, Generator Expression.

---

1. Ev tapşırığının məqsədi

Bu ev tapşırığında Python-da məlumatlarla daha rahat və səmərəli işləməyi məşq edəcəksən.

Artıq `for` dövrləri, `if` şərtləri, list, dictionary, set və funksiyalarla işləməyi bilirsən. Bu dərsdə öyrəndiyimiz mövzular həmin bilikləri daha effektiv istifadə etməyə kömək edir.

İş qaydası: Əvvəlcə 2-ci bölmədəki xatırlatmaları diqqətlə oxu və kod nümunələrini VS Code-da yoxla. Daha sonra 3-cü bölmədəki praktiki tapşırıqları ardıcıllıqla yerinə yetir.

---

2. Mövzuların xatırlatması

Hər kod nümunəsini işlət, nəticəni yoxla və kodun nə etdiyini başa düşməyə çalış.

2.1. List Comprehension

List Comprehension nədir?

Mövcud məlumatlar üzərində dövr keçərək yeni list yaratmağın qısa üsuludur.

Məsələn, mağazada məhsulların adları və qiymətləri saxlanılır.

```
mehsullar = [
    {"ad": "Çay", "qiymet": 6},
    {"ad": "Şokolad", "qiymet": 4},
    {"ad": "Qəhvə", "qiymet": 13},
    {"ad": "Su", "qiymet": 1}
]
```

Bütün məhsulların adlarından ibarət yeni list yaratmaq istəyirik.

Adi `for` dövrü ilə:

```
adlar = []

for mehsul in mehsullar:
    adlar.append(mehsul["ad"])

print(adlar)
```

List Comprehension ilə:

```
adlar = [
    mehsul["ad"]
    for mehsul in mehsullar
]

print(adlar)
```

Nəticə:

```
['Çay', 'Şokolad', 'Qəhvə', 'Su']
```

Kodun izahı:

1. `for mehsul in mehsullar` bütün məhsulların üzərindən keçir.
2. `mehsul["ad"]` hər məhsulun adını götürür.
3. Kvadrat mötərizələr `[]` nəticədə yeni list yaranacağını göstərir.
4. `append()` yazmağa ehtiyac yoxdur. Python yeni listi özü yaradır.

Yadda saxla: List Comprehension mövcud listi dəyişdirmək məcburiyyətində deyil. O, yeni list yaradır.

2.2. List Comprehension ilə filtrləmə

Bəzən bütün məlumatlar deyil, yalnız müəyyən şərtə uyğun olanlar lazımdır.

Məsələn, qiyməti 5 manatdan baha olan məhsulların adlarını seçək.

```
bahali_mehsullar = [
    mehsul["ad"]
    for mehsul in mehsullar
    if mehsul["qiymet"] > 5
]

print(bahali_mehsullar)
```

Nəticə:

```
['Çay', 'Qəhvə']
```

Kodun izahı:

1. `for` bütün məhsulları yoxlayır.
2. `if mehsul["qiymet"] > 5` qiyməti 5 manatdan böyük olanları seçir.
3. Yalnız şərtə uyğun məhsulların adları yeni listə əlavə olunur.

Birdən çox şərt də istifadə etmək mümkündür.

Məsələn, qiyməti 5 manatdan baha və 15 manatdan ucuz olan məhsullar:

```
secili_mehsullar = [
    mehsul["ad"]
    for mehsul in mehsullar
    if mehsul["qiymet"] > 5
    and mehsul["qiymet"] < 15
]

print(secili_mehsullar)
```

Nəticə:

```
['Çay', 'Qəhvə']
```

Yadda saxla: `if` hissəsi `for`-dan sonra yazıldıqda elementləri filtrasiya edir. Şərtə uyğun olmayan elementlər nəticəyə daxil edilmir.

2.3. Comprehension daxilində `if...else`

İndiyə qədər `if` istifadə edərək bəzi elementləri listdən çıxardıq.

Bəs bütün məhsulları saxlamaq, amma qiymətindən asılı olaraq fərqli məlumat göstərmək istəsək?

Məsələn, qiyməti 5 manatdan baha olan məhsullar üçün “Bahalı”, digərləri üçün “Ucuz” yazaq.

```
statuslar = [
    "Bahalı" if mehsul["qiymet"] > 5 else "Ucuz"
    for mehsul in mehsullar
]

print(statuslar)
```

Nəticə:

```
['Bahalı', 'Ucuz', 'Bahalı', 'Ucuz']
```

Kodun izahı:

1. `for` bütün məhsulları yoxlayır.
2. Qiymət 5-dən böyükdürsə, `"Bahalı"` əlavə olunur.
3. Əks halda `"Ucuz"` əlavə olunur.
4. Bütün məhsullar üçün nəticə yaradılır.

Burada iki yazılış arasındakı fərqi yadda saxla.

Filtrasiya:

```
netice = [
    mehsul["ad"]
    for mehsul in mehsullar
    if mehsul["qiymet"] > 5
]
```

Yalnız uyğun məhsulları götürür.

Şərtə görə dəyər seçmək:

```
netice = [
    "Bahalı" if mehsul["qiymet"] > 5 else "Ucuz"
    for mehsul in mehsullar
]
```

Bütün məhsulları nəzərə alır, amma hər biri üçün fərqli dəyər hazırlayır.

2.4. Dictionary Comprehension

Dictionary Comprehension yeni dictionary yaratmaq üçün istifadə olunur.

List comprehension-dan əsas fərqi odur ki, dictionary-də **key və value** saxlanılır.

Məsələn, məhsul adlarını key, qiymətlərini isə value olaraq götürək.

```
qiymetler = {
    mehsul["ad"]: mehsul["qiymet"]
    for mehsul in mehsullar
}

print(qiymetler)
```

Nəticə:

```
{
    "Çay": 6,
    "Şokolad": 4,
    "Qəhvə": 13,
    "Su": 1
}
```

Kodun izahı:

1. Fiqurlu mötərizələr `{}` dictionary yaradıldığını göstərir.
2. `mehsul["ad"]` key olaraq istifadə olunur.
3. `mehsul["qiymet"]` value olaraq istifadə olunur.
4. `for` bütün məhsulları yoxlayır.

Dictionary Comprehension daxilində `if` də istifadə edə bilərik.

```
bahali_qiymetler = {
    mehsul["ad"]: mehsul["qiymet"]
    for mehsul in mehsullar
    if mehsul["qiymet"] > 5
}

print(bahali_qiymetler)
```

Nəticə:

```
{
    "Çay": 6,
    "Qəhvə": 13
}
```

Yadda saxla: Dictionary Comprehension-da key ilə value arasında `:` işarəsi olur.

2.5. Set Comprehension

Set təkrarlanan dəyərləri saxlamayan məlumat strukturudur.

Məsələn, aşağıdakı listdə məhsul kateqoriyaları var.

```
kateqoriyalar = [
    "İçkilər",
    "Şirniyyat",
    "İçkilər",
    "ərzaq",
    "Şirniyyat",
    "ərzaq"
]
```

Bizə yalnız fərqli kateqoriyalar lazımdır.

```
unikal_kateqoriyalar = {
    kateqoriya
    for kateqoriya in kateqoriyalar
}

print(unikal_kateqoriyalar)
```

Nəticədə aşağıdakı dəyərlər olacaq:

```
İçkilər
Şirniyyat
ərzaq
```

Kodun izahı:

1. `for` bütün kateqoriyaların üzərindən keçir.
2. Fiqurlu mötərizələr və key-value cütünün olmaması set yaradıldığını göstərir.
3. Təkrarlanan dəyərlər yalnız bir dəfə saxlanılır.

Vacib: Set daxilində elementlərin sırası fərqli görünə bilər. Bu normaldır.

2.6. Iterable və Iterator

Iterable nədir?

Üzərindən dövr keçmək mümkün olan obyektdir. Məsələn, list, tuple, string və dictionary iterable-dır.

Iterator nədir?

Elementləri bir-bir oxumağa imkan verən və hansı elementdə qaldığını izləyən obyektdir.

Iterator yaratmaq üçün `iter()` istifadə edirik.

```
adlar = ["Çay", "Qəhvə", "Su"]

iterator = iter(adlar)

print(next(iterator))
print(next(iterator))
print(next(iterator))
```

Nəticə:

```
Çay
Qəhvə
Su
```

Kodun izahı:

1. `iter(adlar)` list üçün iterator yaradır.
2. Birinci `next()` ilk elementi qaytarır.
3. İkinci `next()` növbəti elementi qaytarır.
4. Üçüncü `next()` sonuncu elementi qaytarır.
5. Iterator harada qaldığını izləyir.

Bütün elementlər bitdikdə nə baş verir?

Yenidən `next(iterator)` çağırsaq, `StopIteration` xətası yaranacaq.

Bunun qarşısını almaq üçün `next()` funksiyasına ikinci arqument verə bilərik.

```
print(next(iterator, "Element qalmadı"))
```

Nəticə:

```
Element qalmadı
```

İstəsək, `try...except` də istifadə edə bilərik.

```
try:
    print(next(iterator))
except StopIteration:
    print("Iterator bitib.")
```

Vacib: Iterator bir dəfə tam oxunduqdan sonra bitir. Eyni elementləri yenidən oxumaq üçün ilkin iterable-dan yeni iterator yaratmaq lazımdır.

```
yeni_iterator = iter(adlar)

print(next(yeni_iterator))
```

Nəticə:

```
Çay
```

2.7. Generator və `yield`

Generator məlumatları ehtiyac yarandıqca hazırlayıb verən iterator növüdür.

Adi funksiya çox vaxt nəticəni `return` ilə qaytarır.

Generator funksiyasında isə `yield` istifadə edirik.

Məsələn, məhsul adlarını bir-bir verən generator yaradaq.

```
def adlari_getir(mehsul_siyahisi):
    for ad in mehsul_siyahisi:
        yield ad


adlar = ["Çay", "Qəhvə", "Su"]

generator = adlari_getir(adlar)

print(next(generator))
print(next(generator))
```

Nəticə:

```
Çay
Qəhvə
```

Kodun izahı:

1. Funksiya daxilində `yield` istifadə edirik.
2. Funksiyanı çağıranda generator obyekti yaranır.
3. Birinci `next()` çağırıldıqda generator ilk adı verir.
4. Generator həmin yerdə dayanır.
5. Növbəti `next()` çağırıldıqda qaldığı yerdən davam edir.

İndi qalan elementləri oxuyaq.

```
for ad in generator:
    print(ad)
```

Nəticə:

```
Su
```

Niyə yalnız Su gəldi?

Çünki əvvəlki iki elementi artıq `next()` vasitəsilə oxumuşduq.

`return` **və** `yield` **arasındakı fərq:**

- `return` funksiyanın nəticəsini qaytarır və funksiyanı tamamlayır.
- `yield` bir dəyər verir və generatorun daha sonra qaldığı yerdən davam etməsinə imkan yaradır.

Yadda saxla: Generator da iteratordur. Tam oxunduqdan sonra yenidən başlamaq üçün yeni generator yaratmaq lazımdır.

2.8. Generator Expression

Generator yaratmaq üçün hər dəfə ayrıca funksiya yazmaq lazım deyil.

Generator Expression ilə daha qısa şəkildə generator yaratmaq mümkündür.

List Comprehension:

```
reqemler = [1, 2, 3, 4, 5]

kvadratlar = [
    reqem ** 2
    for reqem in reqemler
]

print(kvadratlar)
```

Nəticə:

```
[1, 4, 9, 16, 25]
```

Generator Expression:

```
kvadratlar = (
    reqem ** 2
    for reqem in reqemler
)

print(next(kvadratlar))
print(next(kvadratlar))
```

Nəticə:

```
1
4
```

Burada bütün kvadratları saxlayan list yaratmadıq.

Generator qiymətləri lazım olduqca hesablayır.

Generator Expression `sum()` kimi funksiyalarla da istifadə oluna bilər.

```
reqemler = [10, 20, 30, 40]

cem = sum(
    reqem
    for reqem in reqemler
    if reqem > 15
)

print(cem)
```

Nəticə:

```
90
```

Burada yalnız 15-dən böyük ədədləri götürdük və cəmlədik.

`sum()` generatorun verdiyi dəyərləri bir-bir toplayır.

Yadda saxla:

- `[]` ilə yazılan List Comprehension yeni list yaradır.
- `()` ilə yazılan Generator Expression generator yaradır.
- Generator bütün nəticələri əvvəlcədən ayrıca listdə saxlamır.
- Generator Expression həmişə daha sürətli deyil. Onun əsas üstünlüklərindən biri əlavə yaddaş istifadəsini azaltmaq imkanıdır.

---

3. Praktiki tapşırıqlar

Layihə: Təlim Platformasının Kurs İdarəetmə Sistemi

Təsəvvür et ki, bir təlim mərkəzinin backend sistemini hazırlayırsan.

Bu sistemdə müxtəlif kurslar mövcuddur. Hər kursun adı, kateqoriyası, aylıq qiyməti, boş yer sayı və aktiv olub-olmaması haqqında məlumat saxlanılır.

Sənin məqsədin bu məlumatları emal etmək, uyğun kursları seçmək və nəticələri müxtəlif formalarda hazırlamaqdır.

Aşağıdakı məlumatları Python faylına əlavə et.

```
kurslar = [
    {
        "id": 1,
        "ad": "Python Backend",
        "kateqoriya": "Proqramlaşdırma",
        "qiymet": 180,
        "bos_yer": 4,
        "aktiv": True
    },
    {
        "id": 2,
        "ad": "Frontend",
        "kateqoriya": "Proqramlaşdırma",
        "qiymet": 160,
        "bos_yer": 0,
        "aktiv": True
    },
    {
        "id": 3,
        "ad": "AI Development",
        "kateqoriya": "Süni intellekt",
        "qiymet": 220,
        "bos_yer": 6,
        "aktiv": True
    },
    {
        "id": 4,
        "ad": "UI/UX Design",
        "kateqoriya": "Dizayn",
        "qiymet": 140,
        "bos_yer": 8,
        "aktiv": False
    },
    {
        "id": 5,
        "ad": "Data Analytics",
        "kateqoriya": "Analitika",
        "qiymet": 200,
        "bos_yer": 3,
        "aktiv": True
    },
    {
        "id": 6,
        "ad": "Graphic Design",
        "kateqoriya": "Dizayn",
        "qiymet": 130,
        "bos_yer": 2,
        "aktiv": True
    },
    {
        "id": 7,
        "ad": "Python Basics",
        "kateqoriya": "Proqramlaşdırma",
        "qiymet": 100,
        "bos_yer": 0,
        "aktiv": False
    },
    {
        "id": 8,
        "ad": "Machine Learning",
        "kateqoriya": "Süni intellekt",
        "qiymet": 250,
        "bos_yer": 5,
        "aktiv": True
    }
]
```

Vacib: Bütün tapşırıqlar eyni `kurslar` listi üzərində qurulub. Tapşırıqları bir Python faylında, ayrı-ayrı bölmələr şəklində həll et.

Tapşırıq 1. List Comprehension

Məqsəd: Mövcud dictionary məlumatlarından yeni list yaratmaq.

Tələblər:

1. `kurslar` listindən bütün kursların adlarını götür.
2. `kurs_adlari` adlı yeni list yarat.
3. Mütləq List Comprehension istifadə et.
4. Nəticəni `print()` vasitəsilə göstər.

Gözlənilən nəticə:

```
['Python Backend', 'Frontend', 'AI Development',
 'UI/UX Design', 'Data Analytics', 'Graphic Design',
 'Python Basics', 'Machine Learning']
```

Tapşırıq 2. List Comprehension ilə filtrasiya

Məqsəd: Bir neçə şərtə uyğun kursları seçmək.

Təlim mərkəzi yalnız qeydiyyata açıq kursları göstərmək istəyir.

Kursun qeydiyyata açıq olması üçün iki şərt ödənməlidir:

- `aktiv` dəyəri `True` olmalıdır.
- `bos_yer` sıfırdan böyük olmalıdır.

Tələblər:

1. `qeydiyyata_aciq_kurslar` adlı list yarat.
2. Listdə yalnız şərtlərə uyğun kursların adları olmalıdır.
3. List Comprehension və `if` istifadə et.
4. Hər iki şərti birlikdə yoxla.

Gözlənilən nəticə:

```
['Python Backend', 'AI Development', 'Data Analytics',
 'Graphic Design', 'Machine Learning']
```

Tapşırıq 3. `if...else` ilə kurs statuslarının hazırlanması

Məqsəd: Comprehension daxilində şərtə görə fərqli dəyərlər yaratmaq.

Bu dəfə kursların heç birini siyahıdan çıxarmayacağıq.

Hər kurs üçün qeydiyyat statusu hazırlayacağıq.

Tələblər:

1. `kurs_statuslari` adlı list yarat.
2. Listdə hər element iki məlumatdan ibarət tuple olmalıdır: `(kursun_adı, status)`.
3. Kurs aktivdirsə və boş yeri varsa, status `"Açıq"` olmalıdır.
4. Digər hallarda status `"Bağlı"` olmalıdır.
5. List Comprehension daxilində `if...else` istifadə et.
6. Bütün 8 kurs nəticədə görünməlidir.

Gözlənilən nəticə:

```
('Python Backend', 'Açıq')
('Frontend', 'Bağlı')
('AI Development', 'Açıq')
('UI/UX Design', 'Bağlı')
('Data Analytics', 'Açıq')
('Graphic Design', 'Açıq')
('Python Basics', 'Bağlı')
('Machine Learning', 'Açıq')
```

Tapşırıq 4. Dictionary və Set Comprehension

Məqsəd: Eyni məlumatlardan fərqli Python məlumat strukturları yaratmaq.

1-ci hissə: Dictionary Comprehension

Yalnız aktiv kursların adlarını və qiymətlərini saxlayan dictionary yarat.

Şərtlər:

1. Dictionary-nin key hissəsində kursun adı olsun.
2. Value hissəsində kursun aylıq qiyməti olsun.
3. Yalnız aktiv kurslar daxil edilsin.
4. `aktiv_kurs_qiymetleri` adlı dəyişəndə saxla.

Gözlənilən nəticə:

```
{
    "Python Backend": 180,
    "Frontend": 160,
    "AI Development": 220,
    "Data Analytics": 200,
    "Graphic Design": 130,
    "Machine Learning": 250
}
```

Qeyd: Bu hissədə boş yer şərti yoxdur. Yalnız aktivlik yoxlanılır.

2-ci hissə: Set Comprehension

Sistemdə mövcud olan bütün fərqli kurs kateqoriyalarını əldə et.

Şərtlər:

1. `kateqoriyalar` adlı set yarat.
2. Bütün kursların kateqoriyalarını nəzərə al.
3. Təkrarlanan kateqoriyalar yalnız bir dəfə saxlanılsın.
4. Set Comprehension istifadə et.

Gözlənilən kateqoriyalar:

```
Proqramlaşdırma
Süni intellekt
Dizayn
Analitika
```

Kateqoriyaların sırası fərqli ola bilər.

Tapşırıq 5. Iterator ilə kursların oxunması

Məqsəd: `iter()` və `next()` funksiyalarından istifadə etmək.

Bu tapşırıqda kurs məlumatlarını iterator vasitəsilə bir-bir oxuyacaqsan.

Tələblər:

1. Tapşırıq 1-də yaratdığın `kurs_adlari` listindən istifadə et.
2. Həmin listin ilk üç elementindən yeni list yarat.
3. `iter()` vasitəsilə iterator əldə et.
4. `next()` istifadə edərək üç kursun adını ardıcıl çap et.
5. Dördüncü dəfə `next()` çağır, amma proqram xəta ilə dayanmasın.

6. Element qalmadıqda `"Kurs qalmadı"` mesajı göstər.
7. Eyni üç kurslu listdən yeni iterator yaradaraq ilk kursun adını yenidən çap et.

Gözlənilən nəticə:

```
Python Backend
Frontend
AI Development
Kurs qalmadı
Python Backend
```

İpucu:

- İlk üç elementi `[:3]` vasitəsilə götürə bilərsən.
- `iter()` iterator yaradır.
- `next()` növbəti elementi oxuyur.
- `next(iterator, "Kurs qalmadı")` yazılışı iterator bitəndə xəta əvəzinə mesaj qaytarır.

Tapşırıq 6. Generator funksiyası və `yield`

Məqsəd: Nəticələri bir-bir verən generator funksiyası yaratmaq.

Təlim mərkəzi qeydiyyata açıq olan kursları bir-bir əldə etmək istəyir.

Bu dəfə hazır list qaytaran funksiya yox, generator funksiyası yaradacağıq.

Tələblər:

1. `qeydiyyata_aciq_kurslari_getir(kurs_siyahisi)` adlı funksiya yarat.
2. Funksiya kursların üzərindən `for` dövrü ilə keçsin.
3. Kurs aktivdirsə və boş yeri varsa, həmin kurs dictionary-sini `yield` vasitəsilə versin.
4. Funksiya daxilində yeni nəticə listi yaratma.
5. Funksiyanı çağıraraq generator obyekti əldə et.
6. `next()` vasitəsilə ilk iki uyğun kursun adını çap et.
7. Daha sonra `for` dövrü ilə qalan uyğun kursların adlarını çap et.

Gözlənilən nəticə:

```
İlk iki kurs:
Python Backend
AI Development

Qalan kurslar:
Data Analytics
Graphic Design
Machine Learning
```

İpucu:

- `yield` istifadə et, `return` ilə hazır list qaytarma.
- Generator ilk uyğun kursu verdikdən sonra müvəqqəti dayanır.
- Növbəti `next()` çağırıldıqda qaldığı yerdən davam edir.
- `for` dövrü də generatoru qaldığı yerdən oxuyur.

Uğurlar! *HIGH CODE | Python Backend Development | Lesson 17*